

Exercici 1 (3 punts)

A un laboratori de biotecnologia d'alta seguretat cada dia arriba un conjunt de mostres biològiques $M = \{M_1, \dots, M_n\}$ de diferent origen que cal analitzar abans no es degradin. Cada mostra té una durabilitat de D_i dies, és a dir, només és analitzable qualsevol dia entre el dia de la seva arribada A_i i el dia C_i en què caduca, on $C_i = A_i + D_i$, per a tot $1 \leq i \leq n$.

El laboratori disposa d'una única màquina d'anàlisi i, per tant, només es pot analitzar una mostra al dia. A més, per motius de seguretat biològica, no es poden processar mostres en dies consecutius, ja que fer-ho podria provocar reaccions creuades al sistema.

Donat un conjunt $M = \{M_1, \dots, M_n\}$ de mostres biològiques i les seves dates d'arribada al laboratori $A = \{A_1, \dots, A_n\}$ i respectives durabilitats $D = \{D_1, \dots, D_n\}$, es vol gestionar com dur a terme les anàlisis per tal de poder **maximitzar el nombre de mostres processades**, seleccionant un dia vàlid per a cadascuna de manera que:

1. No es processi més d'una mostra el mateix dia.
2. No es processin mostres en dies consecutius.
3. Cada mostra s'analitzi dins del seu interval de validesa.

Proposeu un algorisme eficient que resolgui el problema. Justifiqueu-ne la correcció i analitzeu-ne el cost.

Una solució:

Formalment, volem maximitzar $|S|$, on la programació S és un subconjunt $S \subseteq \{1, \dots, n\}$ i una assignació $f : S \rightarrow \mathbb{N}$ tal que per a tot $i \in S$, tenim

$$A_i \leq f(i) < C_i, \quad f(i) \neq f(j), \quad \text{i} \quad |f(i) - f(j)| \neq 1, \quad \text{per a tot } i \neq j \in S.$$

Proposem un algorisme golafre. Primer, ordenem les mostres per data de caducitat creixent C_i (en cas d'empat, seleccionem primer la mostra amb A_i més primerenc). A continuació, iterem sobre la llista ordenada i, per a cada mostra M_i , intentem assignar-li el primer dia vàlid $A_i \leq d \leq C_i$ tal que d no s'hagi usat. Si existeix tal d , assignem M_i a d , marquem d com a usat.

La correcció de l'algorisme es basa en l'estratègia golafre. L'algorisme programa mostres en funció de la seva caducitat, triant sempre el dia vàlid més primerenc que no sigui consecutiu. Com que el golafre tria sempre la mostra amb caducitat més propera i el dia vàlid més aviat possible, qualsevol decisió alternativa hauria sacrificat una oportunitat amb menys marge, comproment així el nombre total de mostres possibles. Això implica que cap altra seqüència de decisions pot portar a una solució millor, i per tant, la solució golafre és òptima.

Més formalment, suposem, per contradicció, que existeix una programació millor S^* amb $k + 1$ mostres, mentre que l'algorisme golafre produeix S amb k . Sigui M_i la primera mostra (en el temps) on les programacions difereixen, amb el dia d^* assignat a M_i en S^* (on d^* no havia estat

assignat en el cas de S). Com que el nostre algorisme no va triar d^* , aquest dia ja estava ocupat, era consecutiu a un dia ocupat, o estava fora de l'interval vàlid en el moment de considerar M_i . Per tant, d^* no era una opció vàlida, i S^* no pot ser una programació vàlida. Això contradiu la suposició inicial, i per tant, la solució golafre és òptima.

```

funció PLANIFICAMOSTRES( $A, D, n$ )
   $C \leftarrow [A_i + D_i$  per a cada  $i]$ 
   $M \leftarrow \{(A_i, C_i, i) \mid 1 \leq i \leq n\}$ 
  Ordena  $M$  per  $C_i$  creixent
  DiesUsats  $\leftarrow \emptyset$ 
   $S \leftarrow \emptyset$ 
  per a cada  $(A_i, C_i, i) \in M$  fer
    per  $d = A_i$  fins a  $C_i - 1$  fer
      si  $d \notin \text{DiesUsats}$  aleshores
        DiesUsats  $\leftarrow \text{DiesUsats} \cup \{d - 1, d, d + 1\}$ 
         $S \leftarrow S \cup \{(i, d)\}$ 
        break
      fi si
    fi per
  fi per
  retorna  $S$ 
fi funció

```

Respecte al cost, ordenar les n mostres per data de caducitat costa $O(n \log n)$. Per a cada mostra, podem iterar fins a D_i dies en el pitjor cas. Per tant, el cost total és $O(n \log n + n^2)$.

Exercici 2 (3 punts)

En Ferran Tallacaps està organitzant un banquet per a la reina de Borbònia i els seus convidats, i té l'encàrrec de col·locar-los asseguts en una sola banda d'una llarga taula de banquet. Disposa de la llista dels $2n$ convidats, on cada convidat i té un enter positiu i distint f_i que indica el **favor** que té amb la reina.

La reina demana a en Ferran que col·loqui els convidats de manera **respectuosa**: ella vol seure al **centre** i vol tenir n **convidats a cada costat**, de manera que el **favor dels convidats decreixi monòtonament** a mesura que s'allunyen d'ella; és a dir, qualsevol convidat assegut entre un altre convidat i i la reina ha de tenir un favor més gran que f_i .

A més, en Ferran sap que tots els convidats s'odien entre ells. Per a cada parell de convidats i, j , en Ferran ha quantificat aquest odi amb un enter positiu $h(i, j) = h(j, i)$, que representa l'**odi mutu** entre ells, on $1 \leq i, j \leq 2n$.

Amb tota aquesta informació sobre els $2n$ convidats del banquet, en Ferran us demana que li dissenyeu un algorisme, tan eficient com pugueu, que determini una disposició respectuosa dels convidats i minimitzi la suma total d'odi mutu entre els parells de convidats asseguts un al costat de l'altre. Tingueu en compte que en Ferran necessitarà saber quina és exactament la posició que ha d'ocupar cada convidat. Justifiqueu la correcció i analitzeu el cost del vostre algorisme.

Una solució:

El algoritmo que propongo empieza ordenando los $2n$ convidados en orden creciente de simpatía y utiliza programación dinámica. Para determinar la recurrencia asumo que $f_1 < f_2 < \dots < f_{2n}$.

Si analizamos una solución óptima los $2n$ invitados estarán ubicados n a cada lado, en orden decreciente de afinidad a la reina. En este caso el invitado 1 estará sentado en un extremo y tendremos un invitado $j > 1$ en el otro extremo de la mesa. Notemos que los invitados $2, \dots, j-1$ están sentados a continuación del invitado 1 (en orden) en dirección a la reina. Si eliminamos estos invitados lo que tenemos es una solución óptima del problema de colocar los invitados $j, \dots, 2n$ con n de ellos en el mismo lado que j de manera que el odio total de la configuración sea mínimo. Notemos que en el lado en que no está el invitado j tendremos $2n-j$ invitados y que el invitado $j-1$ no se sienta en el mismo lado que j .

Para poder desarrollar la recurrencia, defino $H(i, j) = \sum_{k=i}^{j-1} h(k, k+1)$, $1 \leq i < j \leq 2n$ y $T(j, \ell)$ como el odio mínimo total de una configuración de los jugadores $j, \dots, 2n$ con ℓ invitados en el lado en que se sienta j , $1 \leq j \leq 2n$ y $0 \leq \ell \leq n$.

Si $\ell = 2n-j+1$, tenemos que colocar a todos los invitados en el mismo lado que j , con lo que $T(j, \ell) = H(j, 2n)$,

Si $\ell < 2n-j+1$, tenemos que decidir quien es el invitado que se colocará lo más alejado de la reina en el lado en el que no se sienta j . Analizando todas las posibilidades tenemos que, en este caso

$$T(j, \ell) = \min_{1 \leq k \leq \ell} \{T(j+k, 2n-l+1) + H(j, j+k-1) + h(j-1, j+k)\},$$

asumiendo que, por simplicidad, $h(0, j) = 0$, para $1 \leq j \leq 2n$. En este caso los invitados j a

$j + k - 1$ se sentarán en orden y el invitado $j + k$ lo hará en el otro lado de la mesa, al lado del invitado $j - 1$.

Podemos definir una tabla auxiliar, $a(i) = h(i, i + 1)$, $1 \leq i < 2n$, y precalcular los valores $A[i] = \sum_{j=1}^i a[j]$. Esta tabla nos permite obtener los valores $H(i, j)$ en tiempo constante.

Ordenar tiene coste $O(n \log n)$, precalcular A usando PD tiene coste $O(n)$. La tabla T tiene $2n^2$ elementos y el coste por elemento es $O(n)$. EL coste del algoritmo es $O(n^3)$ y utiliza espacio adicional $O(n^2)$.

Exercici 3 (2 punts)

Hi ha un diamant preciós que s'exhibeix a un museu només en determinades ocasions. En concret, s'exhibeix durant un conjunt T de m intervals de temps disjunts; és a dir,

$$T = \{(s_i, f_i) \mid 1 \leq i \leq m, s_i < f_i\} \quad \text{i} \quad \forall (s_i, f_i) \in T, (s_j, f_j) \in T : i \neq j : [s_i, f_i] \cap [s_j, f_j] = \emptyset$$

on s_i, t_i són els instants d'inici i acabament de cadascun dels $1 \leq i \leq m$ intervals de temps, respectivament.

El servei de seguretat del museu disposa d'un conjunt $G = \{g_1, \dots, g_n\}$ de guàrdies especialitzats que estan capacitats per vigilar i protegir el preuat diamant. Aquests guàrdies estan molt sol·licitats i la seva feina és molt perillosa, per la qual cosa cada guàrdia $g_i \in G$ només està disposat a vigilar el diamant durant els intervals de temps que especifica a la seva llista L_i . D'aquesta llista, però, no pot treballar més de M_i intervals. Al mateix temps, el contracte que tenen amb el museu especifica que cada guàrdia g_i ha de treballar com a mínim m_i franges horàries.

Dissenyeu un algorisme que decideixi si existeix un possible desplegament de guàrdies que garanteixi que, per a cada interval $(s_i, f_i) \in T$ en què s'exhibeix el diamant, hi hagi almenys un i com a màxim tres guàrdies vigilant-lo. Justifiqueu la correcció i analitzeu el cost del vostre algorisme.

Una solució:

El problema ens demana un assignació de guàrdies a intervals de temps amb restriccions. Plantejaré un model de circulació amb fies inferior per resoldre el problema.

Considerem la xarxa $\mathcal{N} = (G, s, t, c, \ell)$ següent:

- $V = \{s, t\} \cup \{g_1, \dots, g_n\} \cup \{t_1, \dots, t_m\}$ és a dir tindrem un vèrtex per cada guàrdia i per cada interval de temps així com dos vèrtexs addicionals s, t .
- $E = \{(s, g_i) \mid 1 \leq i \leq n\} \cap \{(t_j, t) \mid 1 \leq j \leq m\} \cap \{(g_i, t_j) \mid t_j \in L_i\} \cap \{(t, s)\}$. Connectem s amb cada guàrdia i cada interval amb t , cada guàrdia amb els intervals que vol vigilar i, finalment t amb s per poder tancar la circulació.
- Les capacitats són:
 1. $c(s, g_i) = M_i$ per $1 \leq i \leq n$.
 2. $c(t_j, t) = 3$ per $1 \leq j \leq m$.
 3. $c(g_i, t_j) = 1$ per $j \in L_i, 1 \leq i \leq n$.
 4. La resta d'arestes té capacitat infinit.
- Les fites inferiors són:
 1. $\ell(s, g_i) = m_i$ per $1 \leq i \leq n$.
 2. $\ell(t_j, t) = 1$ per $1 \leq j \leq m$.
 3. La resta d'arestes té fita inferior 0.

Si el problema plantejat té solució, podem assignar valor de flux 1, a les arestes guàrdia interval assignat y completar la assignació de flux per tal de que es compleixen les restriccions de conservació. Notem que al ser una solució valida la assignació de flux serà una circulació a $\mathcal{N}$.

Si tenim una circulació a $\mathcal{N}$ amb valors enters, les arestes (g_i, t_j) amb flux 1, proporcionen una assignació de guàrdies a intervals. Aquesta assignació satisfà les condicions requerides ja que el nombre de intervals assignats a g_i coincideix amb el flux assignat a la aresta (s, g_i) i estarà entre m_i i M_i . El mateix passa per els intervals, tots tindran assignat enter 1 i 3 guàrdies.

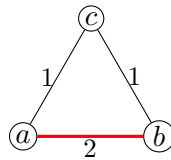
Farem servir el algorisme de basat en Ford Fulkerson per circulacions que té cost $O((D+L)m)$, on D es la suma de les demandes positives, L la suma de les fites inferiors, i m el nombre d'arestes al graf. En el nostre cas $\mathcal{N}$ té $N = n + m + 2$ vèrtexs i $M = n + m + \sum_{i=1}^n |L_i| = O(nm)$ arestes. A més, $L = \sum_{i=1}^n m_i + 3m = O(nm)$ ja que $m_i \leq m$. Per tant el cost de l'algorisme, tenint en compte la construcció de la xarxa i la extracció de l'assignació, té cost $O((nm)^2)$.

Exercici 4a (0.5 punts)

Sigui $G = (V, E, w)$ un graf connex i ponderat. Sigui $U \subseteq V$ tal que el subgraf $G[U]$ induït per U és connex, i sigui T un MST per a $G[U]$ d'acord amb la ponderació w . Si fem servir l'algorisme de Prim començant amb T en comptes de començar amb un vèrtex qualsevol, obtindrem un MST per a G ? Justifiqueu la vostra resposta.

Una solució:

No, començar l'algorisme de Prim amb l'MST T del subgraf $G[U]$ no produeix necessàriament un MST del graf complet G .



Contraexemple: Sigui G un triangle amb els vèrtexs $\{a, b, c\}$ i arestes $w(a, b) = 2$, $w(b, c) = 1$, $w(a, c) = 1$. També, sigui $U = \{a, b\}$. Llavors $G[U]$ és l'aresta (a, b) amb pes 2, així que $T = \{(a, b)\}$. Si comencem l'algorisme de Prim des de T , l'aresta més barata de U cap a $V \setminus U$ pot ser (a, c) amb pes 1. Prim afegirà aquesta aresta i retornarà l'arbre $\{(a, b), (a, c)\}$ amb pes total 3. Però el MST real de G és $\{(a, c), (b, c)\}$ amb pes total 2.

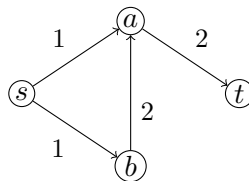
Exercici 4b (0.5 punts)

Digueu si la següent afirmació és certa o falsa i justifiqueu-ne la resposta:

Tenim una xarxa (G, s, t, c) amb $G = (V, E)$ i $c : E \rightarrow \mathbb{Z}^+$. Sigui (A, B) un tall mínim s - t d'aquesta xarxa. Si afegim 1 unitat a totes les capacitats, aleshores (A, B) encara és un tall mínim s - t respecte a les noves capacitats $c(e) + 1$.

Una solució:

Fals. La capacitat d'un tall (A, B) és $C(A, B) = \sum c(e)$, on $e = (u, v) \in E$, $u \in A$, $v \in B$. Si incrementem totes les capacitats en 1, la nova capacitat del tall esdevé $C'(A, B) = \sum (c(e) + 1) = C(A, B) + |\{(u, v) \mid u \in A, v \in B\}|$. Això vol dir que l'increment total depèn del nombre d'arestes que creuen el tall. Un altre tall amb menys arestes creuades pot convertir-se en el nou mínim.



En aquest exemple, el tall mínim s - t pot ser $(\{s\}, \{a, b, t\})$, amb capacitat $1 + 1 = 2$. Després d'incrementar totes les capacitats en 1, la capacitat del mateix tall esdevé $2 + 2 = 4$. Tanmateix,

un altre tall, com $(\{s, a, b\}, \{t\})$, ara té una capacitat menor.

Exercici 4c (0.5 punts)

Digueu si la següent afirmació és certa o falsa i justifiqueu-ne la resposta:

Donat un graf dirigit i ponderat $G = (V, E, w)$ amb $w : E \rightarrow \mathbb{R}$, la manera més ràpida de trobar les distàncies mínimes entre tots els parells de vèrtexs és aplicar n cops l'algorisme de Dijkstra, essent $n = |V|$.

Una solució:

Fals. Aplicar l'algorisme de Dijkstra $n = |V|$ vegades calcula els camins mínims entre tots els parells de vèrtexs en temps $O(n((m+n) \log n))$ (utilitzant una cua amb heap binari). Tanmateix, això només funciona si tots els pesos de les arestes són no negatius. Si el graf conté pesos negatius, l'algorisme de Dijkstra no funciona correctament. En aquests casos, l'algorisme de Floyd-Warshall, que s'executa en temps $O(n^3)$ i gestiona pesos negatius (però no cicles negatius), és l'apropiat.

Fins i tot si tots els pesos són no negatius, en grafs densos ($m = \Theta(n^2)$), el Floyd-Warshall amb cost $O(n^3)$ és millor que executar Dijkstra n vegades, que costa $O(n^3 \log n)$. Tanmateix, si utilitzem l'algorisme de Dijkstra amb una cua de Fibonacci, el cost es redueix a $O(n(m+n \log n)) = O(n^3 + n^2 \log n) = O(n^3)$ i ambdós algorismes empaten asimptòticament.

En grafs dispersos on $m = O(n)$, l'algorisme de Johnson (un Bellman-Ford i n vegades Dijkstra) gestiona pesos negatius i s'executa en temps $O(nm + n^2 \log n)$, que és estrictament millor que Floyd-Warshall.

Exercici 4d (0.5 punts)

Donat un vector A amb n elements, és possible posar en ordre creixent els $\sqrt{n}$ elements més petits i fer-ho en $O(n)$ passos? Si no és possible, expliqueu per què; altrament expliqueu com.

Una solució:

Sí, és possible. Podem trobar i ordenar els $\sqrt{n}$ elements més petits d'un array A amb n elements en temps $O(n)$ utilitzant el procediment següent. Primer, utilitzem un algorisme de selecció en temps lineal (com ara Median of Medians) per trobar l'element de rang $\sqrt{n}$, és a dir, el $\sqrt{n}$ -è més petit, i particionem l'array entorn d'aquest pivot de manera que tots els elements menors o iguals quedin agrupats. Això es fa en temps $O(n)$. A continuació, entre aquests, extraïem els $\sqrt{n}$ elements més petits i els ordenem amb qualsevol algorisme "n-logn" (per exemple, mergesort) en temps $O(\sqrt{n} \log \sqrt{n})$. Com que $O(\sqrt{n} \log \sqrt{n}) = O(\frac{1}{2} \sqrt{n} \log n) = o(n)$, el temps total continua sent $O(n)$.